MINI PROJECT REPORT ON

"Building of a website -VitalCare Healthcare Monitoring System Using Agile Techniques and pipelining"

SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Gyaan Achantani (T11,01)

Devashish Ahuja (T11,02)

Yogini Bhatia (T11,13)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Dr. Sanober shaikh



TABLE OF CONTENTS

Contents	Page Numbers
1. Aim	3
2. Mini project Topic	3
3. Theory	4
4. Implementations	6
5. Lab Outcomes	11
6. Conclusion	12

1. AIM

VitalCare is a full-stack, real-time patient vitals monitoring web application designed to track, triage, and analyze critical patient health metrics (such as blood pressure, oxygen saturation (SpO_2), and heart rate) to assist medical personnel in rapidly identifying at-risk individuals.

The aim of the Vital Care project is to design, implement, and deploy an enterprise-grade, full-stack healthcare application integrated with an automated continuous integration and continuous deployment (CI/CD) pipeline. By combining a robust Spring Boot backend, a responsive React frontend, and a centralized MySQL database deployed via Docker Compose, the platform ensures high availability, modular scalability, and secure data management. Furthermore, the architecture incorporates real-time infrastructure monitoring through Grafana and automated build orchestration via Jenkins, seamlessly linked with Jira issue tracking to enforce rigorous software engineering practices. Ultimately, Vital Care seeks to streamline clinical workflows, eliminate manual deployment overhead, and provide reliable telemetry for resilient healthcare operations.

2. MINI – PROJECT TOPIC

VitalCare: Integrated CI/CD and Real-Time Infrastructure Monitoring Pipeline

The VitalCare project is a full-stack health application engineered to demonstrate modern DevOps practices, automated containerization, and real-time infrastructure observability. Built around a Spring Boot backend and React frontend, the application is orchestrated using Docker Compose to ensure seamless multi-container deployment across local environments.

The application operates through an integrated multi-tier architecture. Healthcare staff interact with a responsive React-based frontend dashboard to monitor patient records categorized into Normal, Watch, and Critical health status. Patient telemetry and demographic data are transmitted to a robust Spring Boot backend service, which executes business logic, evaluates clinical warning thresholds, and persists data securely into a relational MySQL database container.

For operational oversight, monitoring tools like Grafana interface with the data layers to render real-time visual trend graphs. The entire software development lifecycle (SDLC) is managed collaboratively via Agile sprints tracked in Jira, automated through continuous integration pipelines in GitHub Actions and Jenkins, and containerized using Docker Compose to ensure reproducible multi-service deployment across environments.

3. THEORY

The proposed Patient Healthcare Monitoring System is developed using Agile Software Development and DevOps practices to achieve an efficient, reliable, and continuously deliverable software solution. GitHub is used as the source code management platform for maintaining the healthcare application's source code and version history. Developers can commit and push changes to the shared repository, enabling collaborative development and maintaining a centralized version-controlled codebase. GitHub can be integrated with Jenkins so that changes made to the repository can initiate automated software development activities. Docker is used to containerize the Patient Healthcare Monitoring System and package the application together with its dependencies, libraries, configuration, and runtime environment into lightweight and isolated containers.

For a healthcare application consisting of multiple services, Docker Compose is used to define and manage the complete application stack. A Compose configuration can specify application services, databases, networks, ports, environment variables, volumes, and dependencies. It allows multiple containers to be created, started, monitored, stopped, restarted, and removed as a single application environment. This simplifies the deployment and management of the different components of the Patient Healthcare Monitoring System. Grafana is incorporated as a monitoring and visualization platform for the deployed application and its infrastructure. Monitoring provides better visibility into the running system and helps identify performance issues or abnormal behavior at an early stage.

Execution Life Cycle:

1. **User Interaction:** Healthcare workers open the React single-page application (running on port 80 inside Nginx). They view patient cards or submit new health data (e.g., patient name, age, systolic/diastolic blood pressure, SpO_2, heart rate).

2. **API Dispatch:** The React client initiates an HTTP REST call (e.g., POST /api/patients or GET /api/patients) directed to the Spring Boot service running on port 8083:8080.

3. **Backend Processing & Triage Logic:**

    - The REST controller receives the JSON payload and passes it to the Service Layer.

    - The Service Layer applies real-time evaluation logic based on medical risk thresholds:

        - **Critical:** Oxygen Saturation (SpO_2) $< 92\%$, Blood Pressure $> 140/90$, or Heart Rate > 100 bpm.

- **Watch:** SpO_2 between 92%--94% or elevated Heart Rate.
- **Normal:** Vital parameters within standard physiological ranges.

4. **Data Persistence:** Spring Data JPA automatically maps Java objects to MySQL records and executes an SQL INSERT or UPDATE query over a JDBC connection pool into the vitalcare database running in the db container.
5. **Observability & Visual Rendering:**
    - Grafana (port 3000) communicates with the database using a configured native MySQL or Prometheus data source to pull patient metrics.
    - Grafana periodically executes SQL aggregation queries (e.g., SELECT age FROM patients) to plot dynamic graphs and age distribution panels.

Workflow:

A. Agile Project Management (Jira)

- Implementation: The team utilized Jira Software to organize development into structured sprints (e.g., Sprint 0). User stories and tasks—such as containerizing the application, configuring CI/CD pipelines, and developing the backend API—were tracked across custom board columns (To Do, In Progress, In Review, and Done). Jira issues were directly linked to development work items (commits and builds), ensuring end-to-end traceability from planning to execution.

B. CI/CD Automation (GitHub Actions & Jenkins)

- GitHub Actions: Configured workflows (VitalCare CI) to automatically build and test code changes upon pushes or pull request merges to the main branch, ensuring code quality before deployment.
- Jenkins: Implemented a dedicated pipeline (VitalCare-Docker-Test) that automates testing and container verification steps, maintaining stable build histories (#32 to #36).

C. Containerization (Docker & Docker Compose)

- Implementation: To eliminate environment discrepancies, the application was split into four isolated container services managed via Docker Desktop:
    1. frontend: React client exposed on port 80:80.
    2. backend: Spring Boot REST API service running on port 8083:8080.

3. db: MySQL 8.0 relational database instance for data persistence.

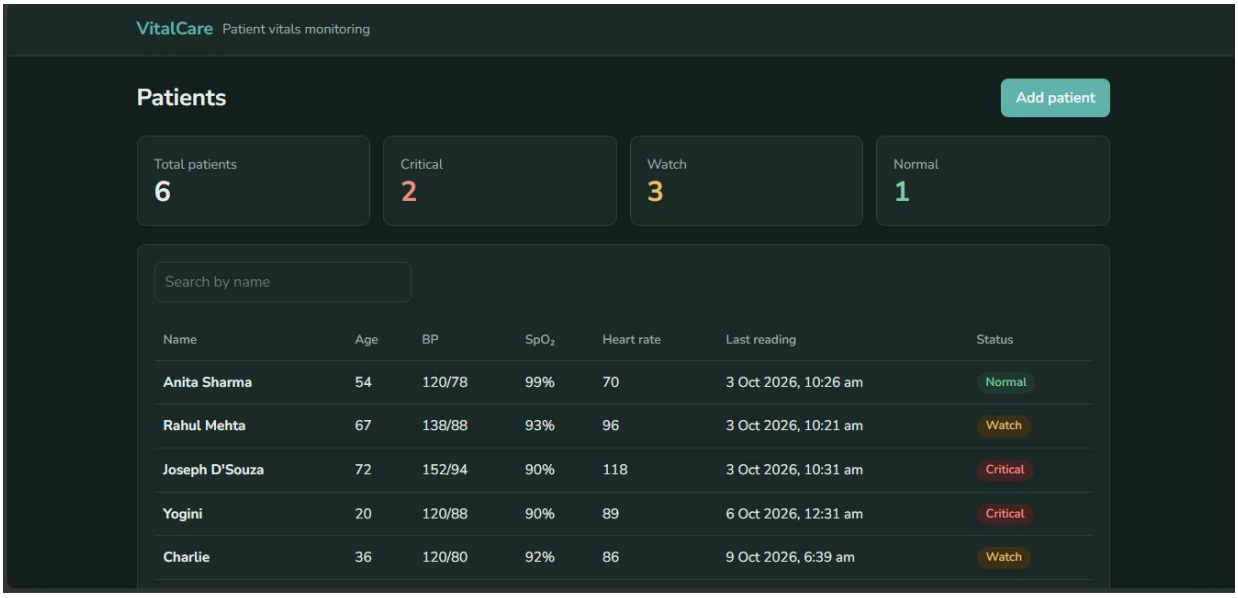
4. grafana: Observability and analytics dashboard server running on port 3000:3000.

These services are orchestrated using a unified docker-compose.yml file, allowing the entire stack to spin up with a single command.

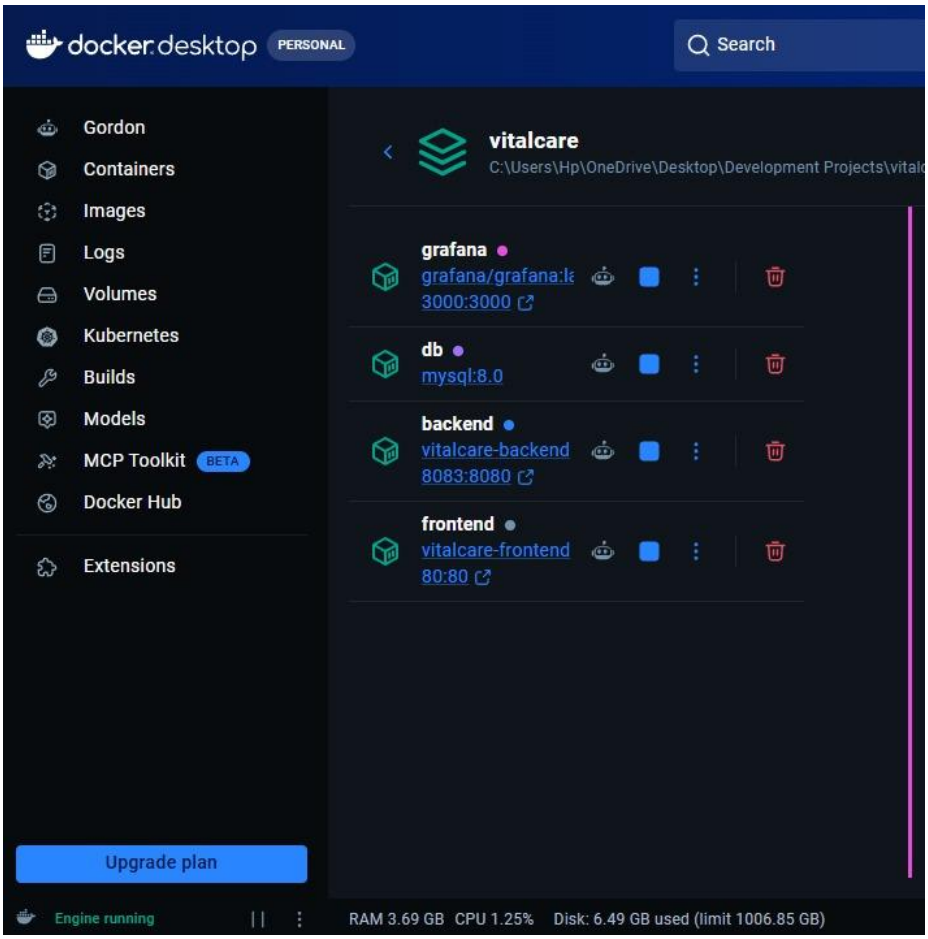
D. Observability & Visualization (Grafana)

- Implementation: Grafana connects to the project's data sources to create specialized dashboards (such as "All patients"). It renders dynamic time-series and parameter curves (e.g., tracking age, heart rate, or vitals distribution over time) for advanced operational telemetry.

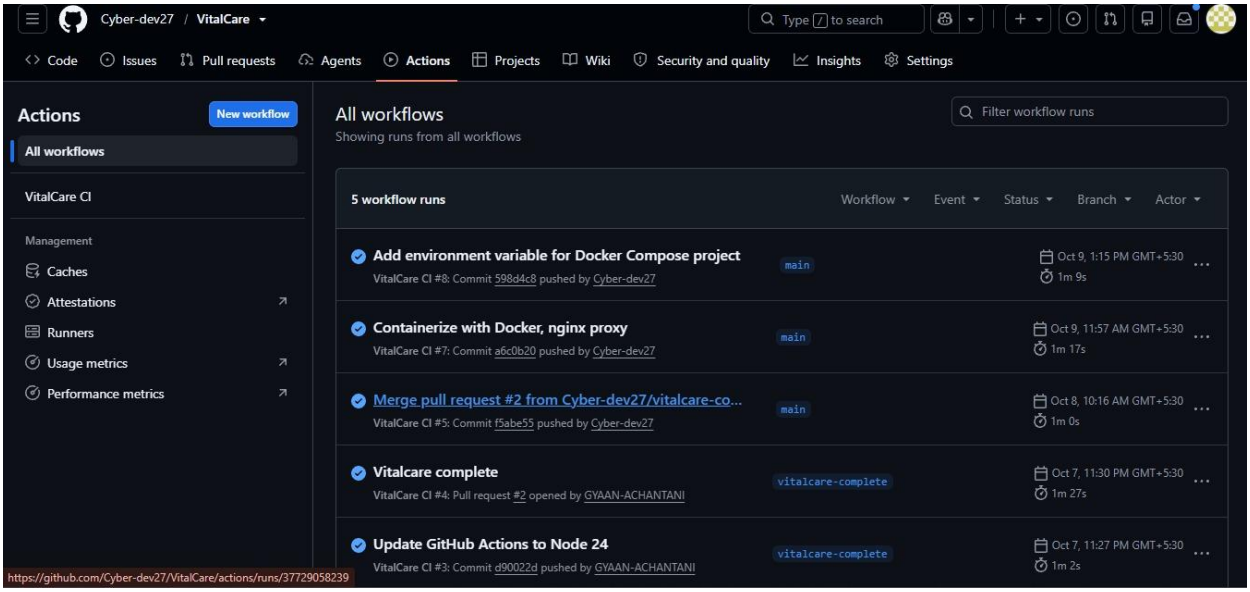
4. IMPLEMENTATIONS



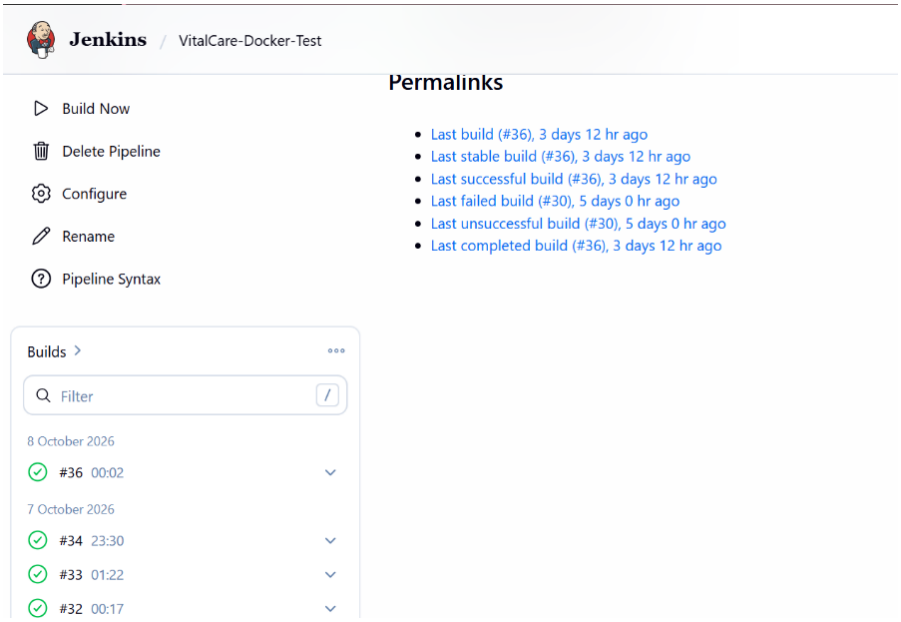
VitalCare Frontend UI: Showcases the primary patient monitoring dashboard displaying aggregate summary cards (Total patients: 6, Critical: 2, Watch: 3, Normal: 1) and a detailed patient vitals table. Shows the main patient monitoring interface with summary stat cards **(Total** patients: 6, Critical: 2, Watch: 3, Normal: 1). Below, a detailed data table lists patient records (Anita Sharma, Rahul Mehta, Joseph D'Souza, Yogini, etc.) with their real-time vitals and assigned triage badges.



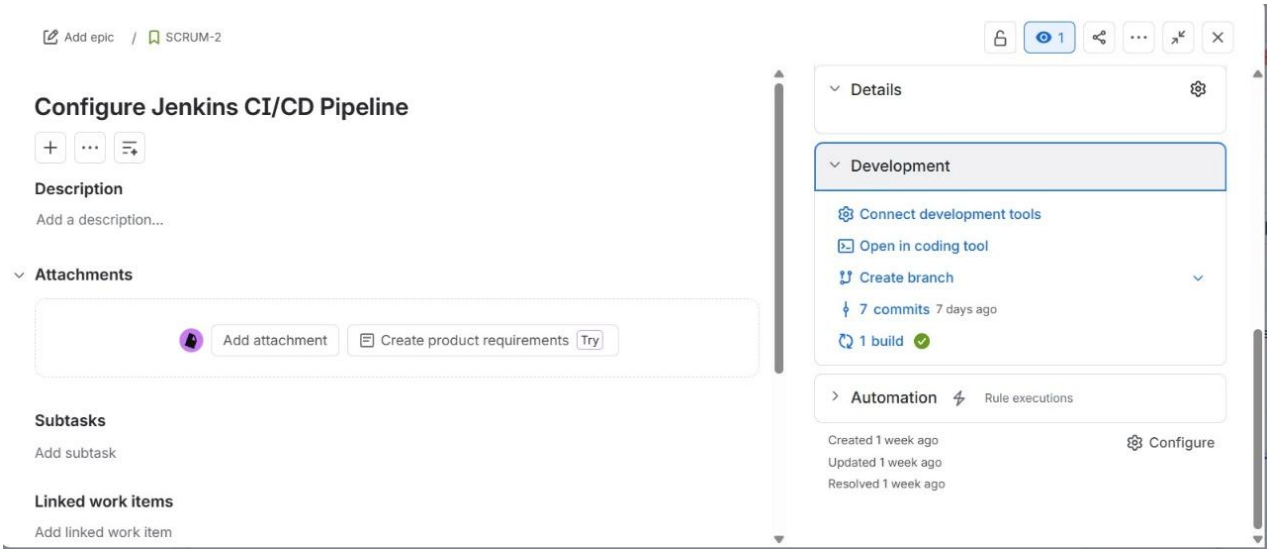
Docker Desktop Dashboard: Visualizes the running multi-container stack (vitalcare), explicitly listing the four active containers: grafana, db (MySQL), backend, and frontend along with their port mappings. Displays the containerized vitalcare environment with four active services: grafana (port 3000), db (MySQL 8.0), backend (8083:8080), and frontend (80:80).



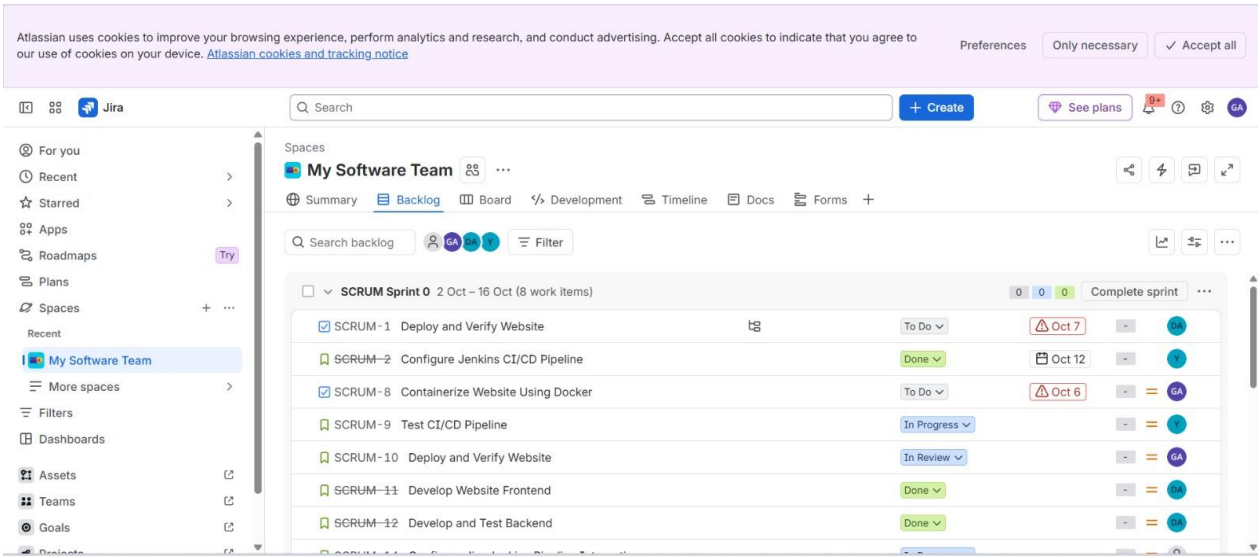
GitHub Actions Dashboard: Displays the automated workflow run history for the VitalCare repository, confirming successful CI pipeline executions for environment variables and containerization. Displays build history for the Cyber-dev27 / VitalCare repository. Shows 5 successful pipeline executions verifying environment configurations and Docker containerization pushes.



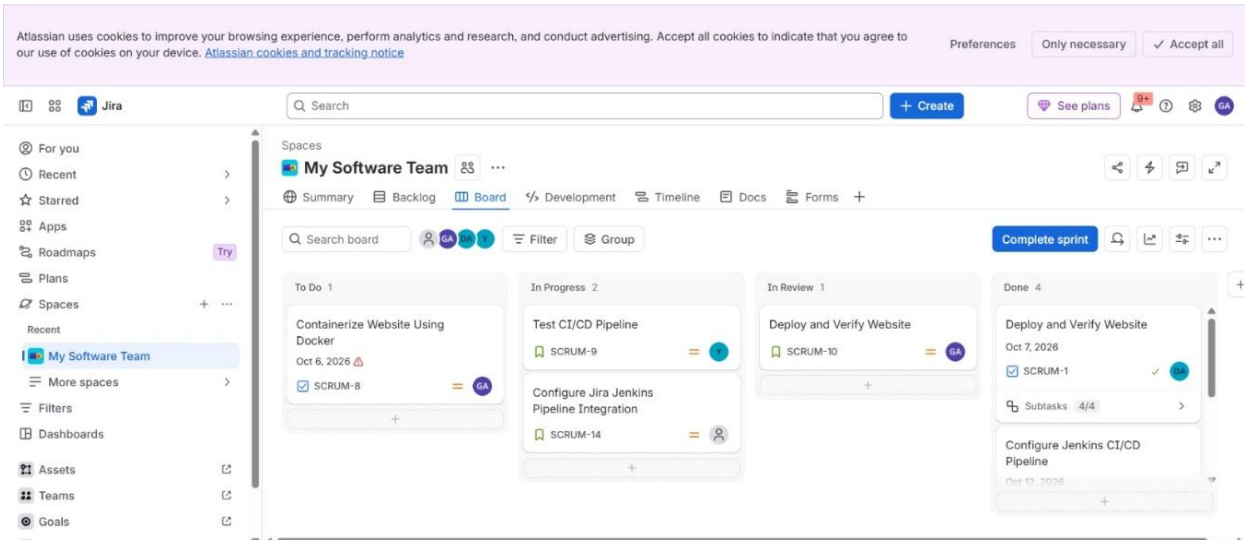
Jenkins Pipeline Dashboard: Shows the execution page for job VitalCare-Docker-Test. Confirms a sequence of passing builds (32 through #36) with complete stability reports.



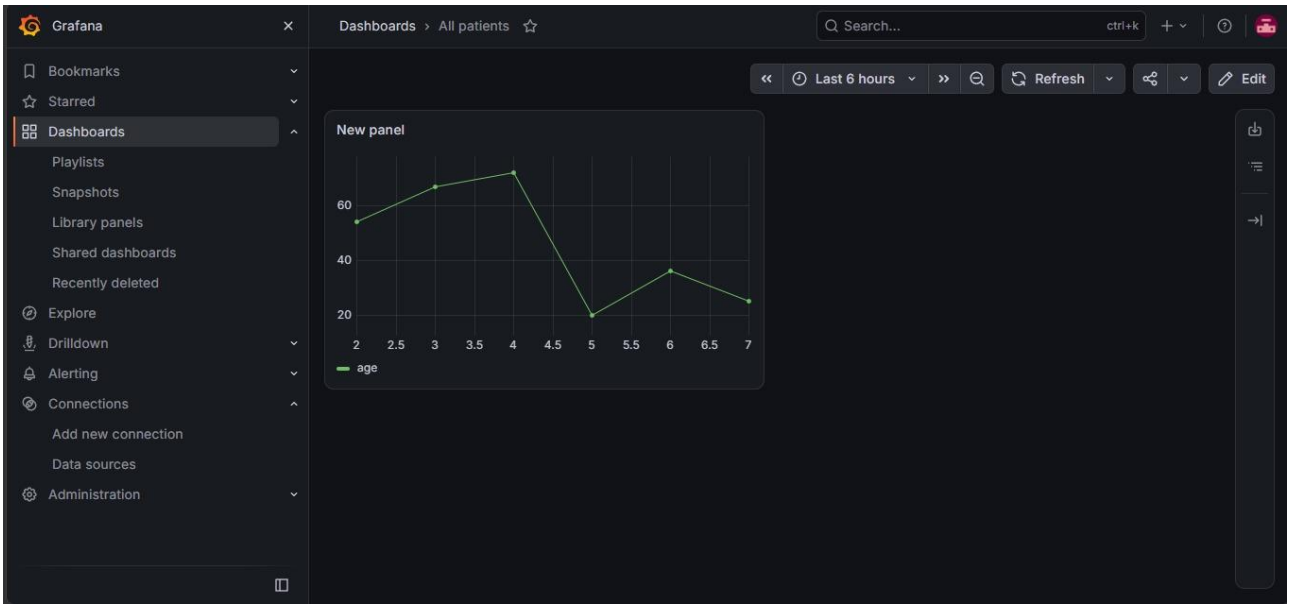
Jira Issue Detail View: Shows a specific task (Configure Jenkins CI/CD Pipeline) linked directly to development tools, recording 7 associated commits and 1 successful build. Highlights task SCRUM-2: Configure Jenkins CI/CD Pipeline. The right panel showcases development integrations, reflecting 7 commits and 1 successful build.



Jira Backlog View: Displays "SCRUM Sprint 0" running from 2 Oct – 16 Oct. It shows work items, priority flags, issue keys, assignees, and target completion dates for application tasks.



Jira Agile Board: Displays the active Scrum board for "My Software Team," organizing project backlog tasks into active workflow stages like To Do (SCRUM-8), In Progress (SCRUM-9, SCRUM-14), In Review (SCRUM-10), and Done (SCRUM-1).



Grafana Monitoring Dashboard: Illustrates the analytics interface ("All patients") with a time-series data panel plotting health metrics and trends. Displays the "All patients" monitoring panel. It renders a visual time-series graph tracking patient metrics (such as age distribution) over a 6-hour window.

As a team, the application was built collaboratively through a coordinated, multi-phase process:

1. **System Architecture & Planning Phase:**
    - Jointly designed the system architecture, defined API request formats, and constructed relational database schemas for patient vitals tracking.
    - Created Jira project boards, defined Sprint 0 timelines, and broke down system requirements into actionable user stories and development tasks.
2. **Full-Stack Development Phase:**
    - Constructed the Spring Boot REST API layer and implemented backend triage rules for analyzing health metrics.
    - Built the React dashboard, developing status badges, search bars, and dynamic tables to communicate with backend endpoints.
    - Configured MySQL database persistence scripts and managed schema migrations.
3. **DevOps, CI/CD, and Observability Integration Phase:**
    - Authored Dockerfile configurations for both frontend and backend services, uniting all layers into a single docker-compose.yml file.
    - Set up GitHub Actions and Jenkins pipelines to automate code checks, Maven package builds, and container integration verification.
    - Configured Grafana dashboards connected to database data sources to visualize live telemetry curves.

5. LAB OUTCOMES

A. LO-03 - Understand and implement Agile Software Development and DevOps practices for software development and deployment.

B. LO-04 - Understand and implement containerization and multi-service application deployment using Docker and Docker Compose.

C. LO-05 - Implement Continuous Integration and Continuous Delivery /Deployment using automation tools such as Jenkins and GitHub.

6. CONCLUSION

The Vital Care project successfully establishes an integrated, containerized full-stack deployment and monitoring architecture. By combining a Spring Boot backend, React frontend, and Grafana monitoring within a Docker Compose environment, the system ensures seamless scalability and reliable performance.

Key Learnings:

- End-to-End SDLC Lifecycle: Gained practical knowledge in connecting sprint tracking in Jira, version control in GitHub, automated testing in Jenkins, container deployment in Docker, and system monitoring in Grafana.
- Containerized Architecture: Mastered multi-tier application orchestration using Docker Compose networking and environment management.
- Continuous Integration Reliability: Learned how automated CI/CD pipelines prevent broken code from reaching deployment environments.
- Agile Team Collaboration: Experienced how structured sprint boards and clear division of labour across frontend, backend, and DevOps enable efficient team project execution.